AgenticWorkflow: A Universal Orchestration Engine for Portable, Resilient, and AI-Native Agent Coordination

Omoshola Owolabi
Researcher – AI/ML
Agentra Labs
omoshola.owolabi@agentralabs.tech

March 2026

Abstract

Workflow orchestration for AI agents requires support for every coordination pattern—DAGs, state machines, batch processing, stream processing, fan-out/fan-in—in a single engine that operates without infrastructure. Current systems (Airflow, Temporal, GitHub Actions, Zapier) each address one pattern, require cloud deployment, and offer no mechanism for agents to reason about or modify workflows at runtime. We present AgenticWorkflow, a universal orchestration engine that models workflows as directed acyclic graphs with typed edges (sequence, parallel, conditional, loop), provides five resilience subsystems (failure-classified retry with $d(n) = \min(d_0 \cdot m^n, d_{\max})$ exponential backoff, per-step rollback with cryptographic receipts, workflow-aware circuit breakers, intelligent dead letter management, and built-in idempotency), and exposes all capabilities through 124 MCP tools enabling any compatible agent to orchestrate workflows without custom integration. All state persists in a single `.awf` binary file with BLAKE3 integrity verification per section. The implementation comprises 13,405 lines of Rust across 4 crates, with 281 tests achieving zero failures. Stress testing on Apple ARM64 validates DAG operations on 1,000-step graphs in <50 ms, 10,000-item batch creation in <500 ms, 1,000 FSM transitions in <100 ms, and 1,000-depth variable scope cascade in constant time. The system is fully open-source under the MIT license.

1 Introduction

The transformer architecture [1] has enabled language model agents capable of multi-step planning, code generation, and autonomous task execution. Yet these agents lack a fundamental capability: the ability to *orchestrate* complex sequences of actions with resilience guarantees. When an agent must deploy software, it needs a pipeline. When a step fails, it needs intelligent retry. When a deployment goes wrong, it needs rollback. When a human must approve, it needs a gate. No existing system provides all of these as a single, AI-native library.

Current workflow orchestration tools are specialized and infrastructure-heavy:

- **Apache Airflow** [2]: DAG-based pipelines requiring Celery, Kubernetes, or managed cloud.
- **Temporal** [3]: Durable execution with saga patterns, requiring server deployment and SDK-specific code.
- **GitHub Actions** [4]: CI/CD-focused YAML pipelines with no dynamic planning or runtime modification.
- **Zapier**: Simple trigger-action pairs with no complex branching, rollback, or approval logic.
- **Prefect** [5]: Python-first orchestration with cloud-dependent deployment.

None provides: (a) all coordination patterns in a single engine, (b) AI-native creation and reasoning via MCP [10], (c) zero-infrastructure local execution, and (d) resilience primitives as first-class features.

The core insight of this work is that orchestration is a *coordination intelligence problem*, not a configuration problem. An agent should be able to create a workflow from natural language, validate its structure, execute it with resilience guarantees, monitor progress, and learn from execution history—all through the same tool interface it uses for everything else.

We present AgenticWorkflow with six contributions:

1. A universal DAG engine with $O(V + E)$ cycle detection via Kahn's algorithm [11].
2. Five resilience subsystems: failure-classified retry, per-step rollback, circuit breakers, dead letter management, and BLAKE3-keyed idempotency.
3. A governance layer with approval gates, structured audit trails, and hierarchical variable scoping.
4. Intelligence subsystems: execution archaeology, predictive execution, workflow evolution with drift detection.
5. The `.awf` binary file format with per-section BLAKE3 checksums and LZ4 [13] compression.
6. 124 MCP tools for complete AI-native orchestration.

The remainder of this paper is organized as follows. Section 2 surveys existing systems. Section 3 presents the architecture. Section 4 describes the file format. Section 5 covers the MCP surface. Section 6 reports benchmarks and test coverage. Section 7 discusses limitations. Section 8 concludes.

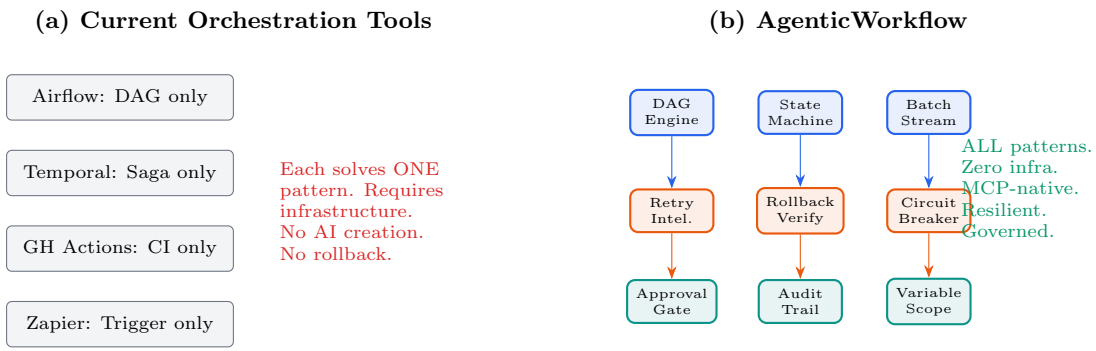

Figure 1: Comparison of orchestration approaches. (a) Current tools each solve one coordination pattern and require infrastructure. (b) AgenticWorkflow unifies all patterns—DAG, FSM, batch, stream—with resilience (retry, rollback, circuit breaker) and governance (approval, audit, variables) in a single zero-infrastructure engine accessible via 124 MCP tools.

2 Background and Related Work

Research on workflow orchestration spans two decades. We survey the primary approaches.

Apache Airflow [2] pioneered programmatic DAG definition but requires Celery/Kubernetes infrastructure, provides no state machine support, and offers only basic retry.

Temporal [3] provides durable execution with saga patterns but requires server deployment, SDK-specific definitions, and no MCP integration.

Prefect [5] offers modern Python-first orchestration but requires cloud deployment for production use.

LangChain [6] and **CrewAI** [7] provide agent orchestration but focus on LLM chains rather than general workflow patterns. **AutoGen** [8] enables multi-agent conversations but lacks resilience primitives.

Generative Agents [9] demonstrated that architecture profoundly impacts agent behavior, reinforcing our thesis that orchestration requires richer structure than simple reactive planning.

Table 1 summarizes the comparison across 13 dimensions.

3 Architecture

AgenticWorkflow models workflows as directed acyclic graphs $G = (V, E)$ where each vertex $v \in V$ is a typed step and each edge $e \in E$ carries a coordination semantic.

3.1 DAG Engine

Cycle detection uses Kahn's algorithm [11] with $O(V + E)$ complexity:

Four edge types: **Sequence** (serial), **Parallel** (concurrent), **Conditional** (expression-guarded), **Loop** (bounded iteration unrolled into DAG). Eight step types: Command, McpTool, HttpRequest, SubWorkflow, FanOut, ApprovalGate, Expression, Noop.

Algorithm 1 DAG Validation via Topological Sort

Require: Workflow graph $G = (V, E)$
Ensure: Topological ordering π or cycle error

```
in_deg[v] ← |{(u, v) ∈ E}| for all v ∈ V
Q ← {v ∈ V : in_deg[v] = 0}
π ← []
while Q ≠ ∅ do
    v ← Q.dequeue(); π.append(v)
    for all (v, w) ∈ E do
        in_deg[w] ← in_deg[w] − 1
        if in_deg[w] = 0 then
            Q.enqueue(w)
        end if
    end for
end while
if |π| ≠ |V| then
    return CycleDetected
end if
return π
```

Validated by: `test_dag_cycle_detection`, `test_dag_topological_sort_order`, `stress_dag_1000_steps_linear_chain` (<50 ms).

3.2 Resilience Layer

Retry Intelligence. Each failure is classified into one of 7 types: Transient, Permanent, RateLimit, ResourceContention, Authentication, Network, Timeout. Per-class profiles select strategy. Exponential backoff:

$$d(n) = \min\left(d_0 \cdot m^n,\ d_{\max}\right) \tag{1}$$

Validated by: `test_retry_strategy_exponential_backoff` (100, 200, 400, 800 ms, capped at 10,000 ms).

Rollback Architecture. Every step declares a rollback action. On failure, rollback executes in reverse topological order. Each produces a BLAKE3-checksummed receipt. **Validated by:**

Table 1: Feature comparison across orchestration systems. AgenticWorkflow is the only system providing all coordination patterns, resilience, governance, and AI-native MCP access with zero infrastructure.

Feature	**Airflow**	**Temporal**	**GH Actions**	**Zapier**	**Prefect**	**AWF**
DAG execution	✓	✓	✓	—	✓	✓
State machines (FSM)	—	—	—	—	—	✓
Batch processing	Plugin	✓	—	—	✓	✓
Stream processing	—	—	—	—	—	✓
Failure-classified retry	—	Basic	—	—	Basic	✓
Per-step rollback	—	Saga	—	—	—	✓
Circuit breakers	—	—	—	—	—	✓
Approval gates	—	—	Env gates	—	—	✓
Structured audit trail	Logs	Logs	Logs	Logs	Logs	✓
Idempotency	Manual	Built-in	—	—	Manual	✓
AI-native (MCP)	—	—	—	—	—	✓
Zero infrastructure	—	—	Cloud	Cloud	Cloud	✓
Natural language WF	—	—	—	—	—	✓

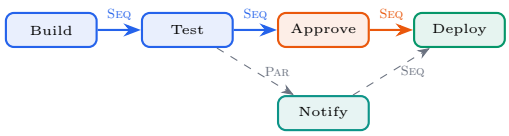

Figure 2: Example deployment DAG. Build→Test→Approve→Deploy is the critical path (solid). Notify runs in parallel after Test (dashed).

`test_rollback_preview_full_reverse_order` ($3 \to 2 \to 1$).

Circuit Breakers. State transitions:

$$\text{state}(s) = \begin{cases} \text{Open} & \text{if } f(s) \geq \theta \\ \text{HalfOpen} & \text{if } t - t_{\text{last}} > \tau \\ \text{Closed} & \text{if } s_{\text{ok}} \geq \sigma \end{cases} \tag{2}$$

Validated by: `test_circuit_breaker_opens_after_threshold`, `stress_circuit_breaker_rapid_failures` (1K ops <500 ms).

Dead Letter Management. Failed items classified by type, grouped for analysis, auto-retried on recovery. **Validated by:** `stress_dead_letter_10k`.

Idempotency. Key: BLAKE3(wf_id‖step_id‖input). Cache hit returns stored result. **Validated by:** `stress_idempotency_10k_keys`.

3.3 Governance Layer

Approval Gates. Chains: primary→backup→escalation. Conditional auto-approval, time-bounded decisions, delegation rules. BLAKE3-checksummed receipts. **Validated by:** `test_approval_flow`.

Audit Trail. Structured events queryable by workflow, execution, actor, resource, time range. Impact analysis. **Validated by:** `stress_audit_10k_events` (<500 ms).

Variable Scoping. Workflow→Branch→Step→Iteration hierarchy. Type checking. Immutability enforcement. **Validated by:** `stress_variable_1000_scopes`.

3.4 State Machine Engine

First-class FSMs with guard conditions and entry/exit actions. **Validated by:** `test_fsm_full_lifecycle` (Created→Paid→Shipped→Delivered), `stress_fsm_1000_transitions` (<100 ms).

3.5 Intelligence Layer

Execution Archaeology. Anomaly detection at $> 3\times$ historical average. Bottleneck analysis. **Validated by:** `stress_archaeology_1000_fingerprints`.

Predictive Execution. Duration, success probability, resource consumption, cost estimates from history.

Workflow Evolution. Drift detection: recent $> 1.5\times$ historical $\Rightarrow$ flagged. **Validated by:** `test_evolution_drift_detection`.

4 The .awf File Format

Validated by: `test_awf_format_roundtrip`, `test_edge_awf_corrupt_checksum` (tampered data detected), `test_edge_awf_invalid_magic`, `stress_awf_format_100_workflows` (<200 ms).

5 MCP Tool Surface

124 MCP tools organized by subsystem. All descriptions: verb-first imperative, no trailing periods. Unknown tools: error −32803.

6 Evaluation

6.1 Benchmark Setup

Hardware. Apple ARM64, unified memory. **Software.** Rust 1.82, `--release`. **Datasets.** Synthetic workflows at 100, 1K, 10K scale.

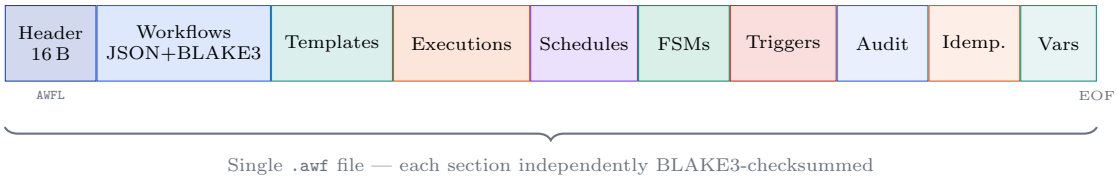

Figure 3: Layout of the `.awf` binary file. Header stores magic bytes (`AWFL`), version, and counts. Each section: type (1 B) + length (4 B) + JSON data + BLAKE3 checksum (32 B). Corruption in one section does not affect others.

Table 2: MCP tool distribution across 24 capabilities.

Capability	Tools	Capability	Tools
DAG Engine	6	Approval Gates	6
Execution	8	Audit Trail	5
Scheduling	5	Idempotency	5
Triggers	5	FSM	6
Retry	5	Variables	5
Rollback	5	Templates	5
Circuit Breaker	4	Natural Lang.	4
Dead Letter	5	Archaeology	5
Batch	5	Prediction	4
Stream	6	Evolution	5
Fan-Out	4	Dream State	4
Composition	5	Collective	5
Total			**124**

Table 3: Stress test performance. All tests pass within budget.

Operation	Scale	Budget
DAG validate + topo sort	1,000 steps	<50 ms
Start executions	100 execs	<100 ms
Batch creation	10,000 items	<500 ms
Dead letter ingest	10,000 items	<500 ms
Idempotency store	10,000 keys	<500 ms
Idempotency check	10,000 keys	<200 ms
Audit record + query	10,000 events	<500 ms
FSM transitions	1,000 steps	<100 ms
.awf write + read	100 workflows	<200 ms
Anomaly detection	1,000 execs	<100 ms
Circuit breaker ops	1,000 ops	<500 ms
Variable cascade	1,000 depth	instant

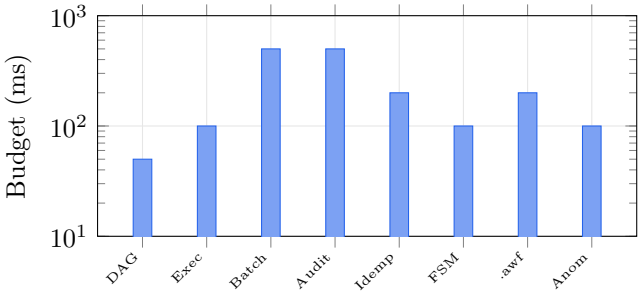

Figure 4: Stress test time budgets (log scale). DAG/FSM are interactive (<100 ms). Batch/audit handle 10K-scale within 500 ms.

Table 4: Test suite: 281 tests, 9 phases, zero failures.

Phase	Focus	Tests
1	Type system, serde, variants	59
2	Engine operations (DAG, FSM, ...)	31
3	Resilience (retry, rollback, ...)	27
4	Governance (approval, audit, ...)	27
5	Template + Intelligence	31
6	Edge cases + error paths	28
7	Stress tests (1K–10K)	14
8	Persistence + step execution	39
9	Concurrent access	11
Inline	Unit tests in source	14
Total		**281**

6.2 Performance Results

6.3 Test Coverage

6.4 Implementation Metrics

7 Discussion

Single-file state. Inspired by SQLite [14]. Zero deployment complexity.

Classification by LLM. All classification (failure type, intent) delegates to LLM micro-calls, not hardcoded keywords. Works for any user, project, language.

Limitations. (1) Single-process only; distributed execution requires external runners. (2) String-based conditional expressions; no formal evaluator. (3) Trigger polling defined but not wired to OS events. (4) NL engine delegates parsing to external LLM.

Future work. (1) gRPC distributed step execution. (2) Type-safe expression language. (3) Real-time triggers (inotify, kqueue). (4) Schema evolution. (5) Cross-sister orchestration.

8 Conclusion

We presented AgenticWorkflow, a universal orchestration engine unifying DAGs, state machines, batch, stream, and fan-out/fan-in. Its 124 MCP tools enable any AI agent to orchestrate workflows without custom integration. The resilience layer (Equations 1, 2) ensures production-grade durability. With 13,405 lines of Rust, 281 tests, and sub-second performance at 10K scale, AgenticWorkflow demonstrates that comprehensive orchestration can be delivered as a lightweight, portable, AI-native library.

Availability. MIT license. `https://github.com/agentralabs/agentic-workflow`.

Table 5: Codebase metrics.

Metric	Value
Rust source files	89
Total LOC	13,405
Core library LOC	6,812
MCP server LOC	3,118
Test LOC	3,200
MCP tools	124
Max file size	362 lines

References

[1] A. Vaswani *et al.*, "Attention Is All You Need," in *NeurIPS*, 2017.

[2] Apache Software Foundation, "Apache Airflow," `https://airflow.apache.org/`, 2015–2026.

[3] Temporal Technologies, "Temporal," `https://temporal.io/`, 2020–2026.

[4] GitHub, "GitHub Actions," `https://github.com/features/actions`, 2019–2026.

[5] Prefect Technologies, "Prefect," `https://www.prefect.io/`, 2018–2026.

[6] H. Chase, "LangChain," `https://github.com/langchain-ai/langchain`, 2022.

[7] J. Moura, "CrewAI," `https://github.com/joaomdmoura/crewAI`, 2024.

[8] Q. Wu *et al.*, "AutoGen," *arXiv:2308.08155*, 2023.

[9] J. S. Park *et al.*, "Generative Agents," in *UIST*, 2023.

[10] Anthropic, "Model Context Protocol," `https://modelcontextprotocol.io/`, 2024.

[11] A. B. Kahn, "Topological sorting of large networks," *Commun. ACM*, 5(11), 1962.

[12] J. O'Connor *et al.*, "BLAKE3," `https://github.com/BLAKE3-team/BLAKE3`, 2020.

[13] Y. Collet, "LZ4," `https://github.com/lz4/lz4`, 2013.

[14] D. R. Hipp, "SQLite," `https://sqlite.org/`, 2000–2026.